

Lab 3: Investigating Process Lifecycles and OS Interaction

I. INTRODUCTION

In Lecture 3, we learned that an OS manages processes and resources. In this lab, we bring that theory to life. Instead of just writing code that prints text, we will write code that interacts with the Linux kernel to query its own process identity, control its execution time, and report its success or failure back to the OS shell.

II. TASK 1: THE LONG-RUNNING PROCESS

OS Insight: In enterprise servers, processes run for days or months. We must know how to monitor them while they are active.

A. C Source Code

We create a program that runs for 30 seconds using a loop and `sleep()`.

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main() {
5     printf("I am starting...\n");
6
7     // Loop for 30 seconds
8     for(int i = 1; i <= 30; i++) {
9         sleep(1); // Pauses execution for 1 second
10    }
11
12    printf("I am finished.\n");
13    return 0;
14 }
```

Listing 1. task1_alive.c

B. Terminal Execution & Verification

To make this interesting, we run the program in the **background** using `&` so we can use a second terminal to inspect it while it's alive.

```
1 # Compile the code
2 gcc task1_alive.c -o task1
3
4 # Run in background (&)
5 ./task1 &
```

Listing 2. Terminal 1: Run in background

```
1 # Find the running process
2 ps aux | grep task1
3
4 # Expected Output:
5 # user 12345 0.0 0.0 2344 560 pts/0 S 10:00 0:00
   ./task1
```

Listing 3. Terminal 2: Monitor the process

III. TASK 2: PROCESS IDENTITY (PID AND PPID)

OS Insight: Every process in Linux has a unique Process ID (PID). The process that created it is the Parent Process ID (PPID), usually your terminal shell.

A. C Source Code

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main() {
5     // Get current Process ID
6     pid_t my_pid = getpid();
7
8     // Get Parent Process ID
9     pid_t my_ppid = getppid();
10
11    printf("My PID is: %d\n", my_pid);
12    printf("My Parent PID is: %d\n", my_ppid);
13
14    printf("Sleeping for 20 seconds...\n");
15    sleep(20);
16
17    return 0;
18 }
```

Listing 4. task2_identity.c

B. Terminal Execution & Verification

```
1 gcc task2_identity.c -o task2
2 ./task2 &
3
4 # Note the PID printed by the program (e.g., 12345)
5 # Verify it using the OS ps command:
6 ps -p 12345 -o pid,ppid,cmd
7
8 # Expected Output:
9 # PID PPID CMD
10 # 12345 8901 ./task2
```

Listing 5. Terminal Commands

IV. TASK 3: EXIT CODES AND OS FEEDBACK

OS Insight: When a program finishes, it returns an integer to the OS. 0 means "Success", and any non-zero number (like 1) means "Error/Failure". The shell stores this in `$?`.

A. C Source Code

```
1 #include <stdio.h>
2
3 int main() {
4     int num;
5     printf("Enter a number (positive for success,
6           negative for fail): ");
7     scanf("%d", &num);
```

```

8  if (num > 0) {
9      printf("Success\n");
10     return 0; // Tell OS: Success
11 } else {
12     printf("Failure\n");
13     return 1; // Tell OS: Error
14 }
15 }

```

Listing 6. task3_exit.c

B. Terminal Execution & Verification

```

1 gcc task3_exit.c -o task3
2
3 # Scenario A: Positive Input
4 ./task3
5 # Enter: 5
6 # Output: Success
7 echo $?
8 # Output: 0
9
10 # Scenario B: Negative Input
11 ./task3
12 # Enter: -5
13 # Output: Failure
14 echo $?
15 # Output: 1

```

Listing 7. Checking Exit Status

V. TASK 4: STANDARD I/O STREAMS

OS Insight: The OS provides standard input (stdin) and standard output (stdout) streams. scanf reads from stdin, and printf writes to stdout.

A. C Source Code

```

1 #include <stdio.h>
2
3 int main() {
4     char name[50];
5
6     printf("Enter your name: ");
7     // Read string from standard input
8     scanf("%s", name);
9
10    printf("Hello, %s! Welcome to OS Class.\n",
11           name);
12    return 0;
13 }

```

Listing 8. task4_input.c

B. Terminal Execution

```

1 gcc task4_input.c -o task4
2 ./task4
3
4 # Terminal Interaction:
5 # Enter your name: John
6 # Hello, John! Welcome to OS Class.

```

Listing 9. Running Task 4

VI. TASK 5: CONDITIONAL EXECUTION AND TERMINATION

OS Insight: Programs often branch based on user input. The OS tracks the final exit code to determine if the script or automation pipeline should continue or halt.

A. C Source Code

```

1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main() {
5     int choice;
6
7     // Print PID at start
8     printf("Current PID: %d\n", getpid());
9
10    printf("Do you want to continue? (1 for Yes, 0 for No): ");
11    scanf("%d", &choice);
12
13    if (choice == 1) {
14        printf("Continuing...\n");
15        sleep(5);
16        return 0; // Success
17    } else {
18        printf("Exiting...\n");
19        return 1; // Failure/Abort
20    }
21 }

```

Listing 10. task5_control.c

B. Terminal Execution & Verification

```

1 gcc task5_control.c -o task5
2
3 # Run 1: Choose Yes
4 ./task5
5 # Current PID: 12400
6 # Do you want to continue? (1 for Yes, 0 for No): 1
7 # Continuing...
8 echo $?
9 # Output: 0
10
11 # Run 2: Choose No
12 ./task5
13 # Current PID: 12405
14 # Do you want to continue? (1 for Yes, 0 for No): 0
15 # Exiting...
16 echo $?
17 # Output: 1

```

Listing 11. Testing Control Flow

VII. CONCLUSION

This lab demonstrated that C programs do not exist in a vacuum; they are active processes managed by the Linux kernel. By utilizing system calls like `getpid()`, `sleep()`, and return codes, we successfully bridged the gap between high-level C code and low-level OS process management.

SUBMISSION CHECKLIST

- All 5 .c source files properly named
- Proper documentaiton of your work, that includes theoretical concepts, flow diagrams and Screenshots of terminal outputs for all verification steps.
- The Code must be pushed to the personal GitHub repository with README.md